

6CM504 Concurrency and communication

Modelling and verification

Prof. Alistair A. McEwan

School of Computing
University of Derby

Table of Contents

- 1 Introduction
- 2 The traces model, and traces refinement
- 3 A reminder about sequences
- 4 Points to ponder
- 5 Summary

This week

Last week, we looked at one of the key aspects of CSP—algebraic laws that allow us to manipulate processes. Many of these laws drew intuition from other areas of mathematics that we are familiar with. However, we did not discuss where these laws come from—for instance, how we can prove they really are laws. Moreover, the laws only allowed us to reason about equivalent processes—they did not allow us to reason about processes that may enjoy different relationships. In this lecture, we shall look at a very powerful aspect of CSP—one of the semantic models that underpins the language.

Table of Contents

- 1 Introduction
- 2 The traces model, and traces refinement**
- 3 A reminder about sequences
- 4 Points to ponder
- 5 Summary

Process traces

- ① A process trace is a record of the sequence of events in which a process has engaged up to a particular point in time
 - Imagine there is a third party observer who writes down the name of each event as it occurs
- ② We denote process traces as a sequence of events—for example
 - The trace $\langle x, y \rangle$ consists of two observation of events—the event x followed by the event y
 - The trace $\langle x \rangle$ is a sequence containing only the event x
 - The trace $\langle \rangle$ is the empty sequence containing no events

Process traces

- 1 We can model a process using the set of all traces that it can perform
- 2 For any process P , $traces(P)$ is the set of all possible traces
- 3 Consider the following example

$$Q = (\begin{array}{l} a \rightarrow b \rightarrow STOP \\ [] \\ c \rightarrow STOP \end{array})$$

- 4 $traces(Q) = \{ \langle \rangle, \langle a \rangle, \langle a, b \rangle, \langle c \rangle \}$
 - The process may have communicated nothing yet $\langle \rangle$
 - It may have communicated the a in the first choice $\langle a \rangle$
 - It may have communicated the b that follows this $\langle a, b \rangle$
 - It may have communicated the c in the second choice $\langle c \rangle$

Working out traces: examples (1)

- 1 Recall the process *STOP*—it does nothing
- 2 The only trace of the process that can perform no event is the empty trace

$$\text{traces}(\text{STOP}) = \{\langle \rangle\}$$

Working out traces: examples(2)

- 1 Another example process involving prefixing but not choice

`Example2 = a -> b -> STOP`

- 2 This process may have communicated nothing yet, it may have performed an `a` only, or it may have performed an `a` then `b`

$\text{traces}(\text{Example2}) = \{\langle \rangle, \langle a \rangle, \langle a, b \rangle\}$

Working out traces: examples(3)

- 1 An example process involving prefixing and choice

`Example3 = a -> STOP [] b -> STOP`

- 2 Here there is a choice of the first event—either a or b
- 3 So there is more than one trace of length 1

$\text{traces}(\text{Example3}) = \{\langle \rangle, \langle a \rangle, \langle b \rangle\}$

Working out traces: examples(4)

- 1 Recursive processes also have traces

`Example4 = a -> Example4 [] b -> STOP`

- 2 This process may perform as many as as it likes, followed by b
- 3 Including no a at all
- 4 Once a b has been observed, the process does nothing more
- 5 This is going to look more complicated to record, but is in fact quite straightforward

$$\text{traces}(Q) = \{ \langle a \rangle^n, \langle a \rangle^n \frown \langle b \rangle \mid n \in \mathbb{N} \}$$

Table of Contents

- 1 Introduction
- 2 The traces model, and traces refinement
- 3 A reminder about sequences**
- 4 Points to ponder
- 5 Summary

Sequences: things to remember(1)

- 1 Note that the process traces are all represented as sets of finite sequences

$$\langle a_1, a_2, \dots, a_n \rangle$$

is the sequence containing a_1, a_2 , to a_n in that order

- 2 Unlike sets, the order is important, as is the number of times an element appears

Sequences: things to remember(2)

- ❶ For instance the sequences:

$\langle a, a, b \rangle$

$\langle a, b \rangle$

$\langle b, a \rangle$

- ❷ are all different—but the sets

$\{a, a, b\}$

$\{a, b\}$

$\{b, a\}$

- ❸ are not—they are exactly the same

Sequences: things to remember

- ① $\langle \rangle$ denotes the empty sequence
- ② For process traces, this means nothing has been observed
- ③ If s and t are two sequences, then $s \frown t$ is their concatenation—the members of s followed by the members of t
 - For instance if $s = \langle a, b \rangle$ and $t = \langle b, a \rangle$
 - Then $s \frown t$ is $\langle a, b, b, a \rangle$

Sequences: things to remember(4)

- 1 If s is a finite sequence and $n \in \mathbb{N}$ then s^n means the n -fold concatenation of s
 - Informally, this means n copies of s concatenated together

$$s^0 = \langle \rangle$$

$$s^{n+1} = s^n \frown s$$

- 2 If s is an initial subsequence of t (known as a prefix of t) in that there is a (possibly empty) sequence w with $t = s \frown w$ then we write $s \leq t$

Sequences and process traces

- ① We will use sets and sequences to describe exactly what a process may be observed to do
- ② This gives us a very precise definition of a process—known as the semantics of the process
- ③ For any process P , $traces(P)$ (the set of all possible traces of the process P) will always have the following properties
 - $trace(P)$ is non-empty: it contains at least the empty trace $\langle \rangle$
 - $trace(P)$ is prefix closed: if $s \frown t$ is a trace of P then at some time in the past the trace was s

Table of Contents

- 1 Introduction
- 2 The traces model, and traces refinement
- 3 A reminder about sequences
- 4 Points to ponder**
- 5 Summary

Point to ponder

- 1 Question: can you calculate the traces of the single shot vending machine

```
OptionVM =  
  coin -> (  
    choc -> STOP  
    []  
    toffee -> STOP )
```

- 2 What about the less user friendly version

```
BadOptionVM =  
  coin -> (  
    choc -> STOP  
    |~|  
    toffee -> STOP )
```

Another point to ponder

- 1 The single shot vending machine is not very popular with customers
- 2 Question: can you calculate the traces of the more practical vending machine

```
FriendlyOptionVM =  
  coin -> (  
    choc -> OptionVM  
    []  
    toffee -> OptionVM )
```

What did we ponder?

- ① Consider the one shot `OptionVM` and `BadOptionVM`
 - What did you notice about their traces?
 - Can you generalise this? What about $P \sqsubseteq Q$ and $P \sqsubseteq^* Q$
 - Does $\text{traces}(P)$ give us a complete description of the behaviours of P ?
 - In fact we will consider this problem later using *more expressive semantic models* of processes—but for now, traces are incredibly important to us
- ② Consider the `FriendlyOptionVM` and the `OptionVM`
 - What did you notice about their traces?
 - Can you generalise this? Does it offer an insight into a relationship between the two processes?

Table of Contents

- 1 Introduction
- 2 The traces model, and traces refinement
- 3 A reminder about sequences
- 4 Points to ponder
- 5 Summary**

Summary

- In this lecture we have been introduced to the traces semantic model of CSP processes—a model which tells us what observations may be made of a process
- We have seen how in many cases, the traces of a process can distinguish it from other processes—but surprisingly, not in all cases
- We have seen how we can manipulate process traces using standard mathematical sequence operators
- In the next lecture, we will look at how we can capture and describe relationships between processes with similar sets of traces

Summary

- Advance reading: Schneider, pages 139–158
- Advance reading: Roscoe, pages 35–47
- Advance reading: Woodcock and Davies, chapter 6